VisionGuard Smart Cane

How the project is built from scratch, stage by stage, with the deep viva questions for each stage



S Darshan (25BCE2883) · Tanay Agarwal (25BCE2475) · BACSE324 Embedded Systems Architecture · Slot E2+TE2

How to read this

The project is built in **14 stages**. Each stage has three parts:

- **What was done:** the actual step, matched to the photos of our cane.
- **Why:** the engineering reason. This is what turns a normal answer into a good one.
- **Deep questions:** what sir may ask at that stage, with answers.

Steps marked "standard practice" are the normal way such projects are tested. Confirm with your teammate which were done exactly that way, and only describe what actually happened.

Stage	Topic	Stage	Topic
1	Problem and requirements	8	Testing each module
2	Concept: shoe → cane	9	Writing the firmware
3	System architecture	10	Calibration
4	Choosing the controller	11	Mechanical layout and mounting
5	Choosing sensors and outputs	12	Wiring and soldering
6	Pin planning and circuit	13	Power circuit
7	Buying parts and setting up the software	14	Final testing and upgrades

Stage 1: Problem and requirements

What was done: identified that a white cane detects obstacles only on contact, gives no distance information and cannot warn about water or drains. Set the requirements: detect obstacles ahead, show distance, detect water at the tip with priority, and stay low-cost and battery-powered.

Why: clear, measurable requirements decide every later choice: the sensor range, the alert type, the power source and the cost limit.

Q. What are your functional and non-functional requirements?

Functional (what it must do): detect obstacles within 50 cm, show distance through the beep rate, detect water and give it priority. **Non-functional** (how well): low cost (under ₹900), light, portable (one battery), simple to use (one switch), fast response.

Q. Is a 50 cm warning distance enough? Justify with numbers.

Walking speed is about 1.2 m/s, so 50 cm gives about 0.4 s. Human reaction to a sound is about 0.25 s, which leaves very little margin. So 50 cm works for a demo, but 100–150 cm is better for real use. It's one constant in the code.

Stage 2: Concept, from smart shoe to cane

What was done: the first design was a smart shoe (Arduino, HC-SR04, soil-moisture sensor, buzzer on D2, vibration motor on D3). It was redesigned as a cane, keeping the same pins.

Why: (1) the cane tip reaches water *before* the foot; (2) a sensor on the shaft is higher and steadier than on a stepping shoe; (3) there's space for the battery and electronics, away from the ground and sweat; (4) blind users already use canes.

Q. What problem would an ultrasonic sensor have on a shoe?

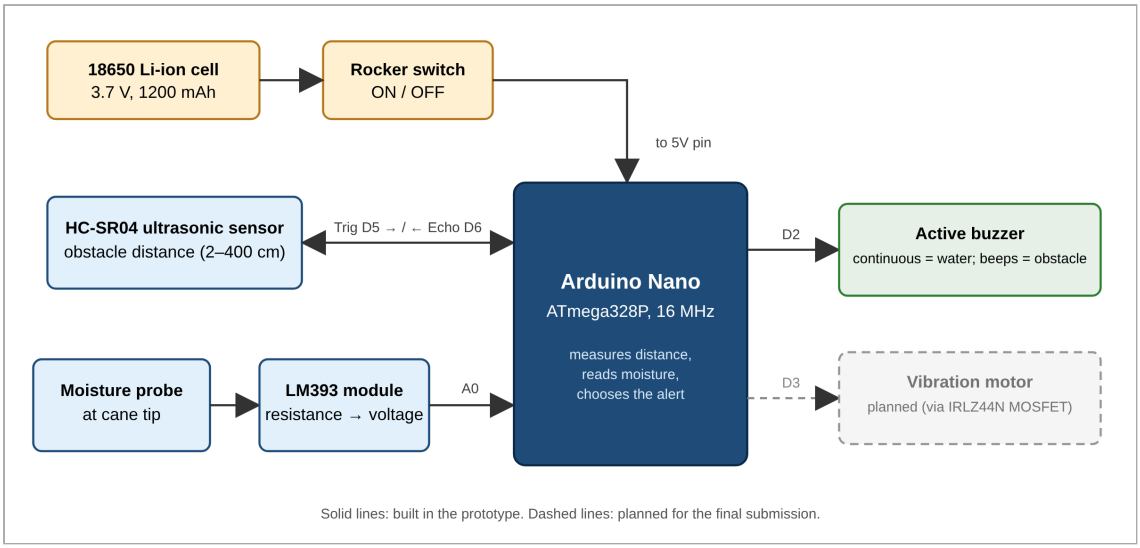
The foot moves up and down and tilts with every step, so the beam keeps pointing at the floor and gives false readings. It's also close to the ground, so dust and water reach it easily.

Q. Where should the sensor be on the cane, and why?

On the shaft, facing forward in the walking direction, high enough that the beam doesn't hit the ground just ahead of the tip. Its angle decides what it sees: tilting it up catches higher obstacles, tilting it down catches lower ones.

Stage 3: System architecture

What was done: drew the block diagram: inputs (ultrasonic sensor, moisture probe through the LM393 module) → processor (Arduino Nano) → output (buzzer), with power from battery → switch → 5V.



Block diagram

Q. What software architecture does your system use?

A **super-loop** (bare-metal): `setup()` runs once, then `loop()` repeats forever, polling the sensors. There's no RTOS. It's a **soft real-time** system: a slightly late alert isn't catastrophic, but it should be quick.

Q. Polling or interrupts: which did you use, and why?

Polling: each loop reads both sensors. It's simple and enough at walking speed. Interrupts would let the CPU react to the echo edges without waiting, which is an improvement for the final version.

Q. Draw the state diagram of your system.

OFF → (switch on) → INIT → SENSE → exactly one of WATER (m < 500), OBSTACLE (m ≥ 500 and 0 < d < 50) or CLEAR → back to SENSE after 30 ms.

Stage 4: Choosing the controller

What was done: chose the **Arduino Nano (ATmega328P, 16 MHz)**, the same chip as the Uno in the shoe design, but only 18 × 45 mm.

Why: small enough for a cane, cheap (about ₹250), 5 V logic like the sensors, a built-in 10-bit ADC for the moisture probe, easy USB programming.

Q. What is the architecture of the ATmega328P?

8-bit AVR **RISC**, **Harvard architecture** (separate program and data memory and buses), 32 general-purpose registers, most instructions in one clock cycle (about 16 MIPS at 16 MHz), and a 2-stage pipeline (fetch + execute).

Q. Name the memories and what each stores in your project.

Flash, 32 KB: the program plus the bootloader. SRAM, 2 KB: variables such as `distance` and `soilMoisture`, and the stack. EEPROM, 1 KB: unused now, but it could store the calibrated threshold.

Q. Why not an ESP32 or a Raspberry Pi?

A Raspberry Pi is a full computer: costly, power-hungry, with slow boot, which is overkill here. An ESP32 uses 3.3 V logic and adds Wi-Fi/Bluetooth we don't need yet. The Nano is the simplest board that does the job.

Stage 5: Choosing sensors and outputs

Part	Chosen	Alternatives	Reason
Obstacle sensor	HC-SR04 ultrasonic	IR sensor, LiDAR	IR is affected by sunlight and object colour and has a short range; LiDAR is costly. Ultrasonic is cheap, 2–400 cm and works in the dark.
Water sensor	Resistive probe + LM393 module	Capacitive sensor, float switch	Cheapest and gives an analog value. Capacitive is better long-term (future scope).
Alert	Active 5 V buzzer	Passive buzzer, speaker, vibration	Beeps with a plain HIGH; vibration is planned.
Battery	18650 Li-ion, 3.7 V, 1200 mAh (IMR type)	9 V battery, AA cells	Rechargeable, light, high energy (4.44 Wh).
Filter	220 µF, 50 V electrolytic	none	Stops supply dips when the buzzer switches.

Q. Why 40 kHz? What limits the HC-SR04 to 2–400 cm?

40 kHz is the transducers' resonant frequency and is above hearing. It's a compromise: higher frequency gives better resolution but is absorbed more by air. The 2 cm minimum comes from **ringing**: the transmitter keeps vibrating briefly after sending, hiding very early echoes. The maximum is set by the echo becoming too weak.

Q. What does the LM393 do? It's a comparator, so why use its analog pin?

The LM393 compares the probe voltage with a reference set by the potentiometer and gives D0 = HIGH/LOW. The module also brings out the raw probe voltage on A0. We read A0 so the threshold is in software and visible on the Serial Monitor.

Q. Active vs passive buzzer: what's the difference electrically?

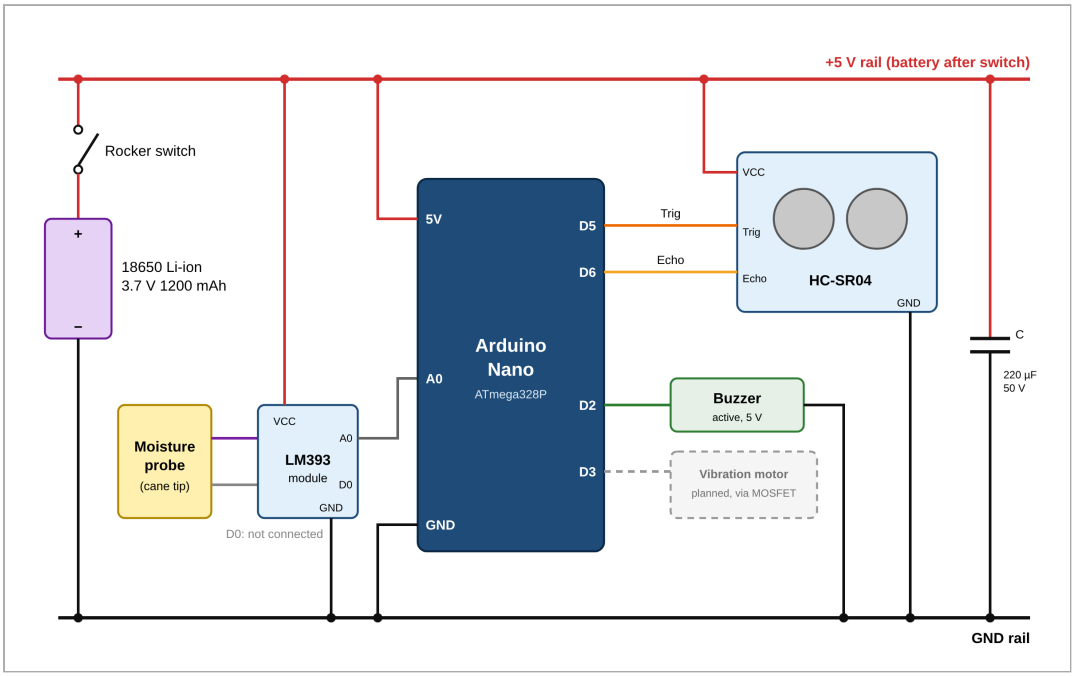
An active buzzer has a built-in oscillator: apply DC and it beeps. A passive buzzer is only a piezo element, so it needs a changing signal (e.g. `tone()`, a square wave) to make sound.

Q. What does "IMR" on the battery mean?

A lithium-manganese (Li-ion manganese oxide) chemistry. It's more thermally stable and safer than older cobalt-based cells, and handles higher current.

Stage 6: Pin planning and circuit

What was done: Trig → D5, Echo → D6, moisture A0 → A0, buzzer → D2, D3 reserved for the vibration motor. All VCCs share the 5V rail, all GNDs a common ground.



Final circuit diagram

Q. Why didn't you use pins D0 and D1?

D0 (RX) and D1 (TX) are the serial pins connected to the USB chip. Using them would interfere with uploading code and with the Serial Monitor.

Q. Why is D3 a good choice for the vibration motor?

D3 supports **PWM** (Timer2), so we can control vibration strength with `analogWrite()`. D2 and D3 are also the external interrupt pins (INT0, INT1), useful if we later add a button.

Q. Why must all grounds be connected together?

Voltages are measured relative to ground. Without a common ground, the Nano can't correctly read the sensors' signal levels.

Stage 7: Buying parts and setting up the software

What was done: bought the parts from Vellore Robo Electronics (invoice 3293, ₹1,735 including GST; cane hardware about ₹876). Installed the Arduino IDE and the CH340 driver, selected Board = Arduino Nano, Processor = ATmega328P (Old Bootloader if needed). *Standard practice: uploaded the Blink example first to test the board.*

Q. Explain the complete path from your code to the chip.

The IDE compiles the C++ with **avr-gcc** into machine code (a .hex file). **avrdude** sends it over USB; the **CH340** converts USB to UART serial; the **bootloader** in the chip's flash receives it and writes it to flash; then the chip resets and runs `setup()`.

Q. What is a bootloader? What does "Old Bootloader" mean?

A small program in flash that lets the chip be programmed over serial without a separate programmer. Older Nanos use an older bootloader at a different upload speed, so the IDE has a separate option for it.

Stage 8: Testing each module *(standard practice)*

What was done: each module is tested on its own before final assembly: the HC-SR04 (distance changes as a hand moves), the moisture sensor (dry about 1023, drops in water) and the buzzer (beeps on HIGH). Values are watched on the Serial Monitor at 9600 baud.

Why: if everything is joined first and it fails, you can't tell which part is wrong. Testing one module at a time isolates faults.

Q. What is a baud rate? Why 9600?

The serial communication speed in bits per second. 9600 is a common, reliable default; we send only a short line each loop, so it's fast enough. The Serial Monitor must be set to the same value.

Q. How did you debug without a display?

With `Serial.print()`: the distance, moisture value and state are printed every loop and watched on the laptop's Serial Monitor.

Stage 9: Writing the firmware

What was done: wrote the program in Arduino C++: name the pins, set them up in `setup()`, then in `loop()` measure distance, read moisture, decide by priority (water → continuous; obstacle → graded beeps; else off) and repeat every 30 ms.

```
unsigned long t = pulseIn(ECHO_PIN, HIGH, 30000); // echo time, 30 ms timeout
int d = t * 0.034 / 2;                             // distance in cm
int m = analogRead(A0);                            // 0–1023
if (m < 500)                  buzzer ON;          // 1. water (priority)
else if (d > 0 && d < 50)   beep, gap = map(d, 2, 50, 50, 400);
else                          buzzer OFF;         // clear
```

Q. What does the CPU do while pulseIn() runs?

It **busy-waits**, looping until the Echo pin changes, so it can't do anything else (it blocks for up to 30 ms). The fix is a pin-change interrupt or Timer1 input capture (pin D8), where hardware records the edge time.

Q. The ATmega328P has no FPU. Is `t * 0.034 / 2` efficient?

No. Floating-point is done in software, which is slower and uses more flash. The integer version `d = t / 58` gives the same result (1 ÷ 0.017 ≈ 58 µs per cm).

Q. What is the worst-case delay before the water alarm sounds?

If water appears just as an obstacle beep starts: 50 ms beep + up to 400 ms gap + 30 ms delay + up to 30 ms pulseIn ≈ **0.5 s**, because `delay()` blocks. Using `millis()` timing would cut it to a few milliseconds.

Q. How is priority implemented, and how could it fail?

Water is the first `if`, so the obstacle check runs only when it isn't wet. It "fails" in practice if the wet reading stays above 500 (threshold not calibrated), or if a very close obstacle's fast beeps sound like the continuous water tone.

Q. Why is the 30 ms timeout important?

The maximum round trip (400 cm) is about 23.5 ms. Without a timeout, pulseIn waits up to 1 second when there's no echo, freezing the cane.

Stage 10: Calibration

What was done: the obstacle distance is set to 50 cm. For water, the dry and wet readings are noted on the Serial Monitor and `WET_SOIL_THRESHOLD` is set between them (default 500).

Q. How do you choose the threshold scientifically?

Take several dry readings and several wet readings (puddle water, wet mud), then set the threshold midway between the lowest dry value and the highest wet value, so both have a safety margin.

Q. The alarm flickers when the value is near 500. Why, and how do you fix it?

Noise moves the reading back and forth across the threshold. Use **hysteresis** (turn on below 500, turn off only above 550) and average a few readings.

Q. How do you reject a single wrong ultrasonic reading?

Use a **median filter**: take 3–5 readings and use the middle value. One stray echo can't change the result.

Stage 11: Mechanical layout and mounting

What was done (see photos): on a white PVC pipe, from the tip upwards:

1. Moisture probe hot-glued at the end, prongs out
2. LM393 module hot-glued about 30 cm up
3. HC-SR04 hot-glued onto a white foam spacer, facing outward
4. Nano hot-glued flat with the USB port reachable; buzzer at its USB end
5. Rocker switch glued below it
6. Battery holder on a foam strip, held by a cable tie

Wires run along the pipe, held with small dots of hot glue.



LM393 and HC-SR04 on the pipe

Q. Why is the heavy part (battery, Nano) near the handle?

It keeps the centre of gravity close to the hand, so the cane is easy to swing and doesn't feel tip-heavy.

Q. Why hot glue? Is it safe on electronics?

Hot glue is a thermoplastic **electrical insulator**, sets in seconds, stays slightly flexible (absorbing vibration) and can be removed for repairs. For a permanent product, screws or a proper enclosure would be stronger.

Q. Why is the HC-SR04 on a spacer?

It lifts the sensor off the curved pipe so it sits flat and points straight out, and keeps the pipe surface out of the beam.

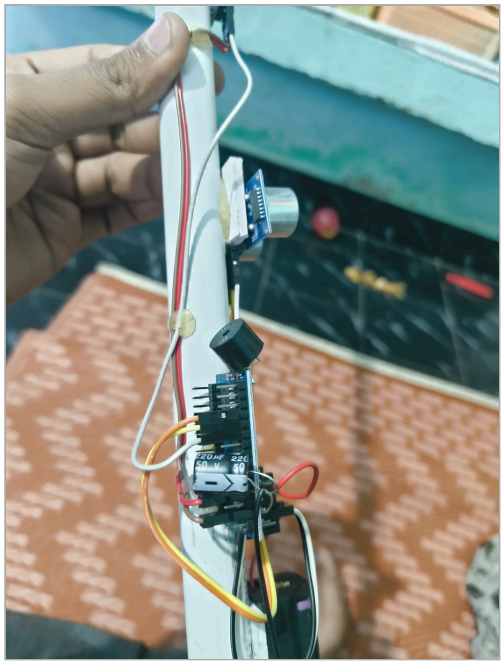
Q. Why PVC pipe?

Light, cheap, strong, easy to glue onto, and an **insulator**, so there's no risk of shorting the circuits.

Stage 12: Wiring and soldering

What was done:

- **Male header pins** soldered onto the Nano.
- Sensors connected with **female jumper wires**: HC-SR04 VCC white, Trig orange → D5, Echo yellow → D6, GND black; LM393 A0 grey → A0, VCC red, GND brown; probe purple + grey.
- **Power wires, buzzer legs and the 220 µF capacitor soldered** directly.
- No breadboard in the final build.



Jumpers on the Nano's header pins and the 220 µF capacitor

Q. Why jumper wires on header pins for signals, but soldering for power?

Jumpers can be unplugged for testing and replacing modules. Power joints carry more current and must never come loose, so they're soldered.

Q. What makes a good solder joint? What is a cold joint?

A good joint is shiny and cone-shaped, with solder flowing onto both the pin and the wire. A **cold joint** is dull and grainy (not enough heat or movement while cooling), with high resistance or an intermittent contact. Tin-lead solder melts at about 183–190 °C; the iron tip is usually 330–370 °C.

Q. What is flux for?

It removes the oxide layer on metal surfaces and helps molten solder spread ("wet") properly, giving a strong joint.

Q. Why use a wire colour code?

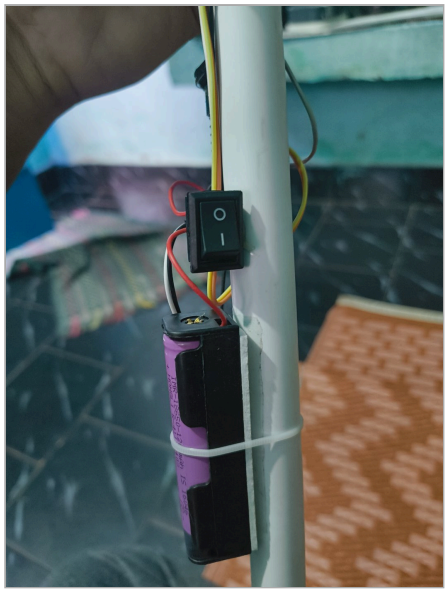
Red for +5 V and black/brown for ground lets you check polarity quickly and avoid reverse connections. Distinct signal colours (orange Trig, yellow Echo) make tracing easy.

Q. Can jumper connections loosen? How do you secure them?

Yes, they're friction-fit. Secure them with a dot of hot glue or heat-shrink, or use locking connectors (like the JST connector on our bill) in the final version.

Stage 13: Power circuit

What was done: battery + (red) → rocker switch → Nano **5V pin**; battery − (black) → GND. The sensors' VCC and GND join these power lines. The 220 µF capacitor is across 5V and GND.



Switch and battery holder

Q. Why the 5V pin and not VIN?

VIN feeds the on-board regulator (an AMS1117-type, which needs about 1 V of headroom), so it needs roughly 6.5–7 V. One 3.7 V cell is too low, so it's connected straight to the 5V rail, bypassing the regulator.

Q. Is 16 MHz guaranteed at 3.7 V?

Not strictly. The datasheet guarantees 10 MHz from 2.7 V and 20 MHz from 4.5 V, roughly linear in between, so 16 MHz needs about 3.8 V. A full cell (4.2 V) is fine, but a drained one is outside the guaranteed range. That's why a 5 V boost converter is our first power upgrade.

Q. Why switch off before connecting USB?

USB 5 V reaches the 5V rail through a diode, and our battery sits directly on that rail. With both connected, USB would push current into the Li-ion cell uncontrolled, which is unsafe.

Q. What protects the battery from over-discharge?

Nothing yet. The chip's brown-out detector (about 2.7 V) only resets the Nano. A TP4056 module with protection (DW01) or a low-battery cut-off in code is needed, since Li-ion shouldn't go below about 3.0 V.

Q. Calculate the battery life.

About 40 mA total. 1200 mAh ÷ 40 mA ≈ 30 h (or 4.44 Wh ÷ 0.15 W). Real life is less.

Stage 14: Final testing and upgrades

What was done: final upload with the switch OFF, then tests on battery: clear path = silent; hand moving from 60 cm to 5 cm = beeps get faster (verified); tip in water = continuous tone (being calibrated).

Q. If the cane stopped working, how would you find the fault?

Check the power first (Nano LED on? battery voltage with a multimeter above 3.5 V?), then the Serial Monitor (are distance and moisture values sensible?), then each jumper and solder joint, and test the module separately.

Q. How would you add a vibration motor correctly?

D3 → IRLZ44N MOSFET gate (a logic-level MOSFET, fully on at 5 V); motor between +V and the drain; source → GND; a **flyback diode** (1N4007) across the motor to absorb the reverse voltage spike when it switches off. A pin alone can't supply motor current (40 mA maximum).

Q. How could only the user hear the alert?

A vibration motor (felt, not heard), a wired earphone using `tone()`, or a bone-conduction/Bluetooth headset with voice messages from a DFPlayer Mini.

Q. What would you change for a real product?

A 5 V boost converter + charger + fuse, a capacitive probe, up/down ultrasonic sensors, vibration feedback, non-blocking code with filtering, and a PCB in a waterproof enclosure.

If you don't know an answer

Say: *"Sir, I'm not fully sure about that detail. My understanding is ___, and I'd confirm it in the datasheet."*
A clear partial answer with a method beats a confident wrong one.